

Arogya-SMC Full Stack Development Environment Setup Ultimate Guide

Welcome to the Arogya-SMC development journey! This guide will take you **from scratch** to set up a complete local development environment containing **a huge amount of realistic mock data**. We will use PostgreSQL/PostGIS as the database, Next.js as the API backend, and finally connect it perfectly with a Flutter frontend.

 **Latest Version:** This guide is based on **PostgreSQL 17**, **PostGIS 3.5**, and **Next.js 15**, following the latest software ecosystem as of March 2026.

Table of Contents

- [Part 1: Install PostgreSQL + PostGIS + pgAdmin](#)
- [Part 2: Initialize Database and Table Structure](#)
- [Part 3: Inject Massive Realistic Mock Data](#)
- [Part 4: Set up Next.js API Backend](#)
- [Part 5: Connect Flutter Frontend](#)
- [Part 6: Screen Recording and Demo Tips](#)

Part 1: Install PostgreSQL + PostGIS + pgAdmin

1.1 Download the Installer

Visit the [PostgreSQL Official Download Page](#) and click "**Download the installer**".

Choose the latest stable version (recommended **PostgreSQL 17**) and download the installer for Windows x86-64.

1.2 Installation Steps

1. **Run the installer** and click **Next**.
2. **Select Components:** Ensure **PostgreSQL Server**, **pgAdmin 4**, **Command Line Tools**, and **Stack Builder** are checked.
3. **Set Password:** Set an **easy-to-remember password** for the **postgres** superuser (you will use this frequently).
4. **Port:** Keep the default **5432**.
5. **Finish Installation:** **Uncheck** "Launch Stack Builder at exit" and click **Finish**.

1.3 Configure Environment Variables

To use the **psql** command in any directory, add the PostgreSQL **bin** folder to your system **Path**:

- Default path: **C:\Program Files\PostgreSQL\17\bin**
- Verify installation: Open a **new** command prompt and run **psql --version**. It should display the version info.

1.4 Install PostGIS using Stack Builder

1. Open the Start menu, find and run **Application Stack Builder**.
2. Select your PostgreSQL installation (e.g., **PostgreSQL 17 on port 5432**) and click **Next**.
3. Expand **Categories** → **Spatial Extensions** in the component tree.
4. **Check PostGIS 3.5 Bundle for PostgreSQL 17** (the version number might vary slightly, just pick the latest).
5. Click **Next**, agree to the license, and follow the steps to finish installation.

 **Stack Builder** is a powerful tool that automatically handles the installation of PostGIS dependencies (like GEOS, PROJ, GDAL), saving you from manual compilation headaches.

1.5 Verify PostGIS Installation

Open **pgAdmin 4**, connect to your local server (using your password). Create a test database and enable PostGIS to verify:

```
CREATE DATABASE test_postgis;
\c test_postgis
CREATE EXTENSION postgis;
SELECT postgis_full_version();
```

If it displays the full version information, the installation was successful.

Part 2: Initialize Database and Table Structure

2.1 Create Arogya-SMC Database

In pgAdmin, right-click **Databases** → **Create** → **Database**:

- **Database:** **arogya_smc**
- **Owner:** **postgres**

Click **Save**.

2.2 Execute Table Creation Scripts

Open the Query Tool for the **arogya_smc** database and run the following SQL scripts in order to create all tables:

```
-- Enable PostGIS extension
CREATE EXTENSION postgis;

-- 1. Administrative Wards Table
CREATE TABLE wards (
    code VARCHAR(10) PRIMARY KEY,
```

```

name VARCHAR(100) NOT NULL,
name_marathi VARCHAR(100), -- Marathi name
name_hindi VARCHAR(100), -- Hindi name
boundary_geojson JSONB, -- GeoJSON for Leaflet maps
population INT, -- Population count
household_count INT, -- Number of households
created_at TIMESTAMP DEFAULT NOW()
);

-- 2. Medical Facilities Table
CREATE TABLE facilities (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    name_marathi VARCHAR(200),
    name_hindi VARCHAR(200),
    ward_code VARCHAR(10) REFERENCES wards(code),
    facility_type VARCHAR(50), -- 'hospital', 'clinic', 'health_center'
    ownership VARCHAR(50), -- 'government', 'private'
    location_lat DECIMAL(10,8),
    location_lng DECIMAL(11,8),
    address TEXT,
    contact_number VARCHAR(20),
    emergency_contact VARCHAR(20),
    has_icu BOOLEAN DEFAULT false,
    has_ventilator BOOLEAN DEFAULT false,
    has_oxygen_plant BOOLEAN DEFAULT false,
    has_ambulance BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 3. Bed Capacity Reports Table
CREATE TABLE capacity_reports (
    id SERIAL PRIMARY KEY,
    facility_id INT REFERENCES facilities(id),
    report_date DATE NOT NULL,
    beds_total INT NOT NULL,
    beds_available INT NOT NULL,
    icu_total INT DEFAULT 0,
    icu_available INT DEFAULT 0,
    ventilators_total INT DEFAULT 0,
    ventilators_available INT DEFAULT 0,
    oxygen_available BOOLEAN DEFAULT false,
    oxygen_cylinders_remaining INT,
    oxygen_estimated_hours INT,
    staff_available INT, -- Number of available medical staff
    created_at TIMESTAMP DEFAULT NOW(),
    submitted_by VARCHAR(100), -- Submitter name
    UNIQUE(facility_id, report_date) -- One report per hospital per day
);

-- 4. ASHA Work Reports Table
CREATE TABLE asha_reports (
    id SERIAL PRIMARY KEY,

```

```

worker_id VARCHAR(50) NOT NULL,
worker_name VARCHAR(100),
ward_code VARCHAR(10) REFERENCES wards(code),
report_date DATE NOT NULL,

-- Symptom Surveillance
fever_count INT DEFAULT 0,
cough_count INT DEFAULT 0,
diarrhea_count INT DEFAULT 0,
jaundice_count INT DEFAULT 0,
rash_count INT DEFAULT 0,
other_symptoms JSONB,          -- Flexible field for other symptoms

-- Maternal Health
pregnant_women_count INT DEFAULT 0,
high_risk_pregnancy_count INT DEFAULT 0,
anc_visits_conducted INT DEFAULT 0,
iron_supplement_distributed INT DEFAULT 0,

-- Child Health
children_under_5_count INT DEFAULT 0,
malnourished_children INT DEFAULT 0,
immunization_due_count INT DEFAULT 0,
immunization_done_count INT DEFAULT 0,

-- Environmental Risks
stagnant_water BOOLEAN DEFAULT false,
poor_sanitation BOOLEAN DEFAULT false,
garbage_dumping BOOLEAN DEFAULT false,
mosquito_breeding BOOLEAN DEFAULT false,

-- Location Information
location_lat DECIMAL(10,8),
location_lng DECIMAL(11,8),
location_accuracy DECIMAL(10,2),

created_at TIMESTAMP DEFAULT NOW(),
sync_status VARCHAR(20) DEFAULT 'pending' -- 'pending', 'synced', 'failed'
);

-- 5. Public Advisories Table
CREATE TABLE advisories (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    title_marathi VARCHAR(200),
    title_hindi VARCHAR(200),
    description TEXT NOT NULL,
    description_marathi TEXT,
    description_hindi TEXT,
    severity VARCHAR(20) NOT NULL, -- 'low', 'medium', 'high', 'critical'
    ward_code VARCHAR(10) REFERENCES wards(code), -- NULL means city-wide
    published_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP,

```

```

    published_by VARCHAR(100),
    views_count INT DEFAULT 0,          -- View count for statistics
    is_active BOOLEAN DEFAULT true
);

-- 6. System Alerts Table
CREATE TABLE alerts (
    id SERIAL PRIMARY KEY,
    type VARCHAR(50) NOT NULL,          -- 'outbreak', 'resource', 'weather',
    'other'
    severity VARCHAR(20) NOT NULL,      -- 'low', 'medium', 'high', 'critical'
    ward_code VARCHAR(10) REFERENCES wards(code),
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    generated_at TIMESTAMP DEFAULT NOW(),
    generated_by VARCHAR(100),          -- Can be 'system' for auto-generated
    acknowledged_at TIMESTAMP,
    acknowledged_by VARCHAR(100),
    resolved_at TIMESTAMP,
    resolved_by VARCHAR(100),
    status VARCHAR(20) DEFAULT 'active' -- 'active', 'acknowledged',
    'resolved'
);

-- 7. Users Table (for Authentication)
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    role VARCHAR(20) NOT NULL,          -- 'admin', 'mho', 'asha', 'hospital',
    'analyst'
    ward_code VARCHAR(10) REFERENCES wards(code), -- For ASHA specifically
    facility_id INT REFERENCES facilities(id),      -- For Hospital users
    specifically
    email VARCHAR(100),
    phone VARCHAR(20),
    is_active BOOLEAN DEFAULT true,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 8. Audit Logs Table
CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id),
    action VARCHAR(50) NOT NULL,        -- 'insert', 'update', 'delete', 'login'
    table_name VARCHAR(50),
    record_id INT,
    old_data JSONB,
    new_data JSONB,
    ip_address INET,
    user_agent TEXT,

```

```

    created_at TIMESTAMP DEFAULT NOW()
);

-- Create indexes to improve query performance
CREATE INDEX idx_capacity_reports_facility_date ON
capacity_reports(facility_id, report_date DESC);
CREATE INDEX idx_asha_reports_ward_date ON asha_reports(ward_code, report_date
DESC);
CREATE INDEX idx_asha_reports_worker_date ON asha_reports(worker_id,
report_date DESC);
CREATE INDEX idx_advisories_ward_active ON advisories(ward_code, is_active,
published_at DESC);
CREATE INDEX idx_alerts_ward_status ON alerts(ward_code, status, generated_at
DESC);
CREATE INDEX idx_facilities_ward ON facilities(ward_code);

```

Part 3: Inject Massive Realistic Mock Data

This section uses PostgreSQL's powerful `generate_series()` function combined with `random()` and SQL tricks to generate **huge amounts of realistic mock data**.

3.1 Generate Ward Data (45 Real Solapur Wards)

```

-- Clear existing data (if running script multiple times)
TRUNCATE wards CASCADE;

-- Insert 45 real ward names
INSERT INTO wards (code, name, name_marathi, name_hindi, population,
household_count, boundary_geojson)
SELECT
    'W' || LPAD(s::text, 2, '0'),
    ward_name,
    ward_name || ' (मराठी)', -- Mock Marathi name
    ward_name || ' (हिंदी)', -- Mock Hindi name
    5000 + (random() * 15000)::int, -- Population 5000-20000
    1000 + (random() * 3000)::int, -- Households 1000-4000
    '{"type":"Polygon","coordinates":[[[75.9,17.68],[75.92,17.68],
[75.92,17.69],[75.9,17.69],[75.9,17.68]]]}'::JSONB
FROM (
    SELECT generate_series(1, 45) AS s,
        CASE (random()*14)::int
            WHEN 0 THEN 'Sadar Bazaar'
            WHEN 1 THEN 'Siddheshwar'
            WHEN 2 THEN 'Railway Colony'
            WHEN 3 THEN 'Hotgi Road'
            WHEN 4 THEN 'Vijapur Road'
            WHEN 5 THEN 'Akkalkot Road'
            WHEN 6 THEN 'Jule Solapur'

```

```

        WHEN 7 THEN 'Kannad'
        WHEN 8 THEN 'Bale'
        WHEN 9 THEN 'Murarji Peth'
        WHEN 10 THEN 'Mangalwar Peth'
        WHEN 11 THEN 'Budhawar Peth'
        WHEN 12 THEN 'Shukrawar Peth'
        WHEN 13 THEN 'Raviwar Peth'
        ELSE 'New Market'
    END || ' Ward' AS ward_name
) AS ward_names;

-- Verify data
SELECT COUNT(*) FROM wards; -- Should return 45

```

3.2 Generate Facility Data (20 Hospitals)

```

-- Clear existing data
TRUNCATE facilities CASCADE;

-- Insert 20 hospitals
INSERT INTO facilities (
    name, name_marathi, name_hindi, ward_code, facility_type, ownership,
    location_lat, location_lng, address, contact_number, emergency_contact,
    has_icu, has_ventilator, has_oxygen_plant, has_ambulance
)
SELECT
    hospital_name,
    hospital_name || ' (मराठी)',
    hospital_name || ' (हिंदी)',
    (SELECT code FROM wards ORDER BY random() LIMIT 1),
    CASE (random()*2)::int
        WHEN 0 THEN 'hospital'
        WHEN 1 THEN 'clinic'
        ELSE 'health_center'
    END,
    CASE (random()*1)::int
        WHEN 0 THEN 'government'
        ELSE 'private'
    END,
    17.6 + (random() * 0.4), -- Latitude range 17.6-18.0
    75.8 + (random() * 0.3), -- Longitude range 75.8-76.1
    'Address ' || s || ', Solapur',
    '98765' || LPAD(s::text, 4, '0'),
    '98765' || LPAD((s+100)::text, 4, '0'),
    random() > 0.3, -- 70% have ICU
    random() > 0.4, -- 60% have Ventilator
    random() > 0.7, -- 30% have Oxygen Plant
    random() > 0.2 -- 80% have Ambulance
FROM (

```

```

SELECT generate_series(1, 20) AS s,
       CASE (random()*4)::int
         WHEN 0 THEN 'Civil Hospital'
         WHEN 1 THEN 'District Hospital'
         WHEN 2 THEN 'Railway Hospital'
         WHEN 3 THEN 'ESI Hospital'
         ELSE 'Community Health Centre'
       END || ' ' || s AS hospital_name
) AS hospital_names;

```

3.3 Generate Capacity Reports (90 Days * 20 Hospitals = 1800+ records)

```

-- Clear existing data
TRUNCATE capacity_reports CASCADE;

-- Generate 90 days of history
INSERT INTO capacity_reports (
    facility_id, report_date, beds_total, beds_available,
    icu_total, icu_available, ventilators_total, ventilators_available,
    oxygen_available, oxygen_cylinders_remaining, oxygen_estimated_hours,
    staff_available, created_at
)
SELECT
    f.id,
    CURRENT_DATE - (s * INTERVAL '1 day') AS report_date,
    beds_base.beds_total,
    -- Available beds fluctuate over time for realism
    GREATEST(0, beds_base.beds_total * (0.2 + random() * 0.5))::int AS
beds_available,
    f.has_icu::int * (5 + (random() * 15)::int) AS icu_total,
    f.has_icu::int * (random() * 8)::int AS icu_available,
    f.has_ventilator::int * (2 + (random() * 8)::int) AS ventilators_total,
    f.has_ventilator::int * (random() * 5)::int AS ventilators_available,
    CASE WHEN random() > 0.2 THEN true ELSE false END AS oxygen_available,
    CASE WHEN random() > 0.3 THEN (10 + (random() * 90)::int) ELSE NULL END AS
oxygen_cylinders,
    CASE WHEN random() > 0.3 THEN (5 + (random() * 48)::int) ELSE NULL END AS
oxygen_hours,
    10 + (random() * 30)::int AS staff_available,
    NOW() - (random() * INTERVAL '90 days')
FROM facilities f
CROSS JOIN generate_series(0, 89) AS s
CROSS JOIN LATERAL (
    SELECT (50 + (random() * 200)::int) AS beds_total
) AS beds_base;

-- Verify data volume
SELECT COUNT(*) FROM capacity_reports; -- Should be around 1800

```


3.4 Generate ASHA Work Reports (50 Workers * 30 Days = 1500+ records)

```
-- Clear existing data
TRUNCATE asha_reports CASCADE;

-- Generate ASHA reports
INSERT INTO asha_reports (
    worker_id, worker_name, ward_code, report_date,
    fever_count, cough_count, diarrhea_count, jaundice_count, rash_count,
    other_symptoms,
    pregnant_women_count, high_risk_pregnancy_count, anc_visits_conducted,
    iron_supplement_distributed,
    children_under_5_count, malnourished_children, immunization_due_count,
    immunization_done_count,
    stagnant_water, poor_sanitation, garbage_dumping, mosquito_breeding,
    location_lat, location_lng, location_accuracy,
    created_at, sync_status
)
SELECT
    'ASHA' || LPAD(w.worker_num::text, 3, '0') AS worker_id,
    'ASHA Worker ' || w.worker_num AS worker_name,
    (SELECT code FROM wards ORDER BY random() LIMIT 1) AS ward_code,
    CURRENT_DATE - (s * INTERVAL '1 day') AS report_date,

    -- Symptom counts (occasionally high for trend simulation)
    CASE
        WHEN random() < 0.1 THEN 20 + (random() * 30)::int -- 10% chance of
outbreak
        ELSE (random() * 10)::int
    END AS fever_count,
    (random() * 8)::int AS cough_count,
    (random() * 5)::int AS diarrhea_count,
    (random() * 2)::int AS jaundice_count,
    (random() * 3)::int AS rash_count,
    jsonb_build_object('other', (random() * 5)::int) AS other_symptoms,

    -- Maternal data
    (1 + (random() * 5)::int) AS pregnant_women,
    (random() * 2)::int AS high_risk,
    (random() * 4)::int AS anc_visits,
    (random() * 10)::int AS iron_distributed,

    -- Child data
    (10 + (random() * 30)::int) AS children_under_5,
    (random() * 5)::int AS malnourished,
    (random() * 8)::int AS immunization_due,
    (random() * 10)::int AS immunization_done,

    -- Environment risks
    random() > 0.7 AS stagnant_water,
```

```

random() > 0.6 AS poor_sanitation,
random() > 0.8 AS garbage_dumping,
random() > 0.7 AS mosquito_breeding,

-- Location
17.6 + (random() * 0.4) AS lat,
75.8 + (random() * 0.3) AS lng,
5 + (random() * 15) AS accuracy,

NOW() - (random() * INTERVAL '30 days') AS created_at,
CASE WHEN random() > 0.1 THEN 'synced' ELSE 'pending' END AS sync_status
FROM (
    SELECT generate_series(1, 50) AS worker_num
) w
CROSS JOIN generate_series(0, 29) AS s;

```

3.5 Generate Public Advisories

```

-- Clear existing data
TRUNCATE advisories CASCADE;

-- Insert advisories
INSERT INTO advisories (
    title, title_marathi, title_hindi, description, description_marathi,
    description_hindi,
    severity, ward_code, published_at, expires_at, published_by, is_active
)
VALUES
    ('Dengue Alert - Use Mosquito Nets',
    'डेंग्यू अलर्ट - डासांच्या जाळ्या वापरा',
    'डेंग्यू अलर्ट - मच्छरदानी का उपयोग करें',
    'Multiple dengue cases reported in your area. Use mosquito repellents and
    remove stagnant water immediately.',
    'आपल्या भागात डेंग्यूचे अनेक रुग्ण आढळले आहेत. डास प्रतिबंधक वापरा आणि साचलेले पाणी
    त्वरित काढून टाका.',
    'आपके क्षेत्र में डेंगू के कई मामले सामने आए हैं। मच्छर भगाने वाली दवा का उपयोग करें और
    जमा पानी तुरंत हटाएं।',
    'high', NULL, NOW() - INTERVAL '5 days', NOW() + INTERVAL '10 days', 'MHO
    Solapur', true),

    ('COVID-19 Booster Drive',
    'कोविड-१९ बूस्टर मोहीम',
    'कोविड-१९ बूस्टर अभियान',
    'Booster doses available at all government hospitals. Walk-ins welcome.',
    'सर्व सरकारी रुग्णालयात बूस्टर डोस उपलब्ध. थेट येऊन घेऊ शकता.',
    'सभी सरकारी अस्पतालों में बूस्टर डोस उपलब्ध। सीधे आ सकते हैं।',
    'medium', NULL, NOW() - INTERVAL '2 days', NOW() + INTERVAL '20 days',
    'SMC Health Dept', true),

```

```

('Chikungunya Outbreak - Sadar Bazaar',
'चिकनगुनियाचा उद्रेक - सदर बाजार',
'चिकनगुनिया का प्रकोप - सदर बाजार',
'Sudden spike in chikungunya cases in Sadar Bazaar. Report fever
immediately.',
'सदर बाजार भागात चिकनगुनियाच्या रुग्णांमध्ये अचानक वाढ. ताप आल्यास त्वरित कळवा.',
'सदर बाजार क्षेत्र में चिकनगुनिया के मामलों में अचानक वृद्धि। बुखार होने पर तुरंत सूचित
करें।',
'high', 'W01', NOW() - INTERVAL '1 day', NOW() + INTERVAL '7 days',
'Epidemiologist', true),

('Vaccination Camp - Railway Colony',
'लसीकरण शिबिर - रेल्वे कॉलनी',
'टीकाकरण शिविर - रेल्वे कॉलनी',
'Free vaccination camp for children under 5 at Railway Colony Health
Centre.',
'रेल्वे कॉलनी आरोग्य केंद्रात ५ वर्षाखालील मुलांसाठी मोफत लसीकरण शिबिर.',
'रेल्वे कॉलनी स्वास्थ्य केंद्र में ५ वर्ष से कम उम्र के बच्चों के लिए मुफ्त टीकाकरण शिविर।',
'low', 'W03', NOW() - INTERVAL '3 days', NOW() + INTERVAL '5 days',
'ICDS', true),

('Heatwave Advisory',
'उष्णतेच्या लाटेचा सल्ला',
'हीटवेव एडवाइजरी',
'Temperatures rising. Stay hydrated and avoid afternoon sun.',
'तापमान वाढत आहे. भरपूर पाणी प्या आणि दुपारच्या उन्हात राहणे टाळा.',
'तापमान बढ़ रहा है। खूब पानी पिएँ और दोपहर की धूप में रहने से बचें।',
'medium', NULL, NOW(), NOW() + INTERVAL '15 days', 'SMC', true);

-- Add more auto-generated advisories
INSERT INTO advisories (
    title, description, severity, ward_code, published_at, expires_at,
    published_by, is_active
)
SELECT
    'Health Advisory ' || s,
    'Automated health advisory message number ' || s || ' for testing
    purposes.',
    CASE (random()*2)::int
        WHEN 0 THEN 'low'
        WHEN 1 THEN 'medium'
        ELSE 'high'
    END,
    CASE WHEN random() > 0.5 THEN (SELECT code FROM wards ORDER BY random()
    LIMIT 1) ELSE NULL END,
    NOW() - (random() * INTERVAL '30 days'),
    CASE WHEN random() > 0.3 THEN NOW() + (random() * INTERVAL '30 days') ELSE
    NULL END,
    'System',
    random() > 0.1
FROM generate_series(1, 20) AS s;

```

3.6 Generate System Alerts

```
-- Clear existing data
TRUNCATE alerts CASCADE;

-- Generate alerts
INSERT INTO alerts (
    type, severity, ward_code, title, description, generated_at, status
)
SELECT
    CASE (random()*2)::int
        WHEN 0 THEN 'outbreak'
        WHEN 1 THEN 'resource'
        ELSE 'weather'
    END AS type,
    CASE (random()*3)::int
        WHEN 0 THEN 'low'
        WHEN 1 THEN 'medium'
        WHEN 2 THEN 'high'
        ELSE 'critical'
    END AS severity,
    (SELECT code FROM wards ORDER BY random() LIMIT 1) AS ward_code,
    CASE (random()*2)::int
        WHEN 0 THEN 'Disease outbreak detected'
        WHEN 1 THEN 'Resource shortage alert'
        ELSE 'Weather warning'
    END || ' ' || s AS title,
    'Automated alert generated by system at ' || NOW() - (random() * INTERVAL
'7 days') AS description,
    NOW() - (random() * INTERVAL '7 days') AS generated_at,
    CASE
        WHEN random() < 0.3 THEN 'resolved'
        WHEN random() < 0.6 THEN 'acknowledged'
        ELSE 'active'
    END AS status
FROM generate_series(1, 50) AS s;
```

3.7 Create Test Users

```
-- Note: In real apps, passwords should be bcrypt hashed. Using plain text here
for simplicity.
INSERT INTO users (username, password_hash, full_name, role, ward_code,
facility_id, email, phone, is_active)
VALUES
    ('mho', 'smc123', 'Dr. Kulkarni', 'admin', NULL, NULL, 'mho@solapur.gov',
'9876543210', true),
    ('civil', 'hospital123', 'Civil Hospital Admin', 'hospital', NULL, 1,
```

```
'civil@hospital.gov', '9876543211', true),
  ('asha01', 'asha123', 'Meena Jadhav', 'asha', 'W01', NULL,
'meena.j@asha.org', '9876543212', true),
  ('asha02', 'asha123', 'Sunita Pawar', 'asha', 'W02', NULL,
'sunita.p@asha.org', '9876543213', true),
  ('analyst', 'analyst123', 'Rajesh Sharma', 'analyst', NULL, NULL,
'rajesh@solapur.gov', '9876543214', true);
```

Part 4: Set up Next.js API Backend

4.1 Create Next.js Project

```
# Create project (select default settings)
npx create-next-app@latest arogya-backend
cd arogya-backend

# Install dependencies
npm install pg jsonwebtoken bcryptjs cookie
npm install -D @types/pg @types/node # TypeScript type support

# Install shadcn/ui (Optional, for admin UI)
npx shadcn@latest init
```

4.2 Configure Database Connection

Create `lib/db.ts`:

```
// lib/db.ts
import { Pool } from 'pg';

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'arogya_smc',
  password: process.env.DB_PASSWORD || 'your_password_here', // Replace with
your password
  port: parseInt(process.env.DB_PORT || '5432'),
  max: 20, // Max connection pool size
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

export default pool;
```

Create `.env.local`:

```
DB_USER=postgres
DB_PASSWORD=your_password_here
DB_NAME=arogya_smc
DB_HOST=localhost
DB_PORT=5432
JWT_SECRET=your_jwt_secret_key_here_change_in_production
```

4.3 Create Public API Routes

Create `app/api/public/facilities/route.ts`:

```
import { NextRequest, NextResponse } from 'next/server';
import pool from '@/lib/db';

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const ward = searchParams.get('ward');

  try {
    let query = `
      SELECT
        f.id, f.name, f.name_marathi, f.name_hindi,
        f.ward_code, w.name as ward_name,
        f.location_lat, f.location_lng,
        f.address, f.contact_number, f.emergency_contact,
        f.facility_type, f.ownership,
        f.has_icu, f.has_ventilator, f.has_oxygen_plant, f.has_ambulance,
        cr.beds_total, cr.beds_available,
        cr.icu_total, cr.icu_available,
        cr.ventilators_total, cr.ventilators_available,
        cr.oxygen_available,
        cr.report_date as last_updated,
        EXTRACT(EPOCH FROM NOW() - cr.report_date) / 3600 as hours_since_update
      FROM facilities f
      LEFT JOIN wards w ON f.ward_code = w.code
      LEFT JOIN LATERAL (
        SELECT * FROM capacity_reports cr
        WHERE cr.facility_id = f.id
        ORDER BY cr.report_date DESC
        LIMIT 1
      ) cr ON true
    `;

    const params: any[] = [];

    if (ward) {
```

```

    query += ` WHERE f.ward_code = $1`;
    params.push(ward);
  }

  query += ` ORDER BY f.name`;

  const result = await pool.query(query, params);

  // Add availability status
  const facilitiesWithStatus = result.rows.map((facility: any) => ({
    ...facility,
    bed_status: facility.beds_available / facility.beds_total > 0.3 ? 'good'
:
    facility.beds_available / facility.beds_total > 0.1 ?
'warning' : 'critical',
    is_recent: facility.hours_since_update < 24
  }));

  return NextResponse.json(facilitiesWithStatus);
} catch (error) {
  console.error('Database error:', error);
  return NextResponse.json(
    { error: 'Internal Server Error' },
    { status: 500 }
  );
}
}

```

Create `app/api/public/advisories/route.ts`:

```

import { NextRequest, NextResponse } from 'next/server';
import pool from '@/lib/db';

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const ward = searchParams.get('ward');
  const lang = searchParams.get('lang') || 'en'; // 'en', 'mr', 'hi'

  try {
    let query = `
      SELECT
        id,
        CASE
          WHEN $1 = 'mr' AND title_marathi IS NOT NULL THEN title_marathi
          WHEN $1 = 'hi' AND title_hindi IS NOT NULL THEN title_hindi
          ELSE title
        END as title,
        CASE
          WHEN $1 = 'mr' AND description_marathi IS NOT NULL THEN
description_marathi

```

```

        WHEN $1 = 'hi' AND description_hindi IS NOT NULL THEN
description_hindi
        ELSE description
        END as description,
        severity,
        ward_code,
        published_at,
        expires_at,
        published_by
    FROM advisories
    WHERE is_active = true
        AND (expires_at IS NULL OR expires_at > NOW())
    `;

const params: any[] = [lang];

if (ward) {
    query += ` AND (ward_code = $2 OR ward_code IS NULL)`;
    params.push(ward);
}

query += ` ORDER BY
    CASE severity
        WHEN 'critical' THEN 1
        WHEN 'high' THEN 2
        WHEN 'medium' THEN 3
        WHEN 'low' THEN 4
    END,
    published_at DESC`;

const result = await pool.query(query, params);
return NextResponse.json(result.rows);
} catch (error) {
    console.error('Database error:', error);
    return NextResponse.json(
        { error: 'Internal Server Error' },
        { status: 500 }
    );
}
}

```

4.4 Create Login API

Create `app/api/auth/login/route.ts`:

```

import { NextRequest, NextResponse } from 'next/server';
import pool from '@/lib/db';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

```



```

const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key';

export async function POST(request: NextRequest) {
  try {
    const { username, password } = await request.json();

    // Query user
    const result = await pool.query(
      `SELECT id, username, password_hash, full_name, role, ward_code,
facility_id
      FROM users WHERE username = $1 AND is_active = true`,
      [username]
    );

    if (result.rows.length === 0) {
      return NextResponse.json(
        { error: 'Invalid credentials' },
        { status: 401 }
      );
    }

    const user = result.rows[0];

    // Verify password (simplified for example, use bcrypt.compare in
    production)
    const isValid = password === user.password_hash;

    if (!isValid) {
      return NextResponse.json(
        { error: 'Invalid credentials' },
        { status: 401 }
      );
    }

    // Generate JWT
    const token = jwt.sign(
      {
        id: user.id,
        username: user.username,
        role: user.role,
        ward_code: user.ward_code,
        facility_id: user.facility_id
      },
      JWT_SECRET,
      { expiresIn: '1d' }
    );

    // Update last login
    await pool.query(
      'UPDATE users SET last_login = NOW() WHERE id = $1',
      [user.id]
    );
  }
}

```

```

const response = NextResponse.json({
  success: true,
  role: user.role,
  name: user.full_name
});

// Set cookie
response.cookies.set('token', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: 60 * 60 * 24, // 1 day
  path: '/',
});

return response;
} catch (error) {
  console.error('Login error:', error);
  return NextResponse.json(
    { error: 'Internal Server Error' },
    { status: 500 }
  );
}
}

```

4.5 Create Auth Middleware

Create `middleware.ts`:

```

import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { jwtVerify } from 'jose';

const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key';

export async function middleware(request: NextRequest) {
  const path = request.nextUrl.pathname;
  const token = request.cookies.get('token')?.value;

  // Public paths
  const publicPaths = [
    '/',
    '/api/public',
    '/api/auth/login',
    '/_next/static',
    '/favicon.ico',
  ];

```

```

if (publicPaths.some(p => path.startsWith(p))) {
  return NextResponse.next();
}

// Paths requiring auth
if (!token) {
  return NextResponse.redirect(new URL('/', request.url));
}

try {
  // Verify JWT
  const secret = new TextEncoder().encode(JWT_SECRET);
  await jwtVerify(token, secret);

  return NextResponse.next();
} catch (error) {
  // Invalid token
  return NextResponse.redirect(new URL('/', request.url));
}

export const config = {
  matcher: [
    '/api/:path*',
    '/dashboard/:path*',
    '/hospital/:path*',
  ],
};

```

Part 5: Connect Flutter Frontend

5.1 Add Dependencies in Flutter

Add these to your `pubspec.yaml`:

```

dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  provider: ^6.1.1 # State management
  shared_preferences: ^2.2.2 # Local storage
  intl: ^0.19.0 # Date formatting

```

5.2 Create API Service Class

```

// lib/services/api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  // Choose correct baseUrl based on environment
  static const String baseUrl = 'http://10.0.2.2:3000/api/public'; // Android
Emulator
  // static const String baseUrl = 'http://localhost:3000/api/public'; // iOS
Emulator
  // static const String baseUrl = 'http://192.168.1.5:3000/api/public'; //
Physical device

  // Fetch facilities list
  static Future<List<dynamic>> fetchFacilities({String? wardCode}) async {
    try {
      final url = wardCode != null
        ? '$baseUrl/facilities?ward=$wardCode'
        : '$baseUrl/facilities';

      final response = await http.get(Uri.parse(url));

      if (response.statusCode == 200) {
        return json.decode(response.body);
      } else {
        throw Exception('Failed to load facilities: ${response.statusCode}');
      }
    } catch (e) {
      throw Exception('Network error: $e');
    }
  }

  // Fetch advisories
  static Future<List<dynamic>> fetchAdvisories({String? wardCode, String lang =
'en'}) async {
    try {
      String url = '$baseUrl/advisories?lang=$lang';
      if (wardCode != null) {
        url += '&ward=$wardCode';
      }

      final response = await http.get(Uri.parse(url));

      if (response.statusCode == 200) {
        return json.decode(response.body);
      } else {
        throw Exception('Failed to load advisories');
      }
    } catch (e) {
      throw Exception('Network error: $e');
    }
  }
}

```

```
// Fetch ward list
static Future<List<dynamic>> fetchWards() async {
  try {
    final response = await http.get(Uri.parse('$baseUrl/wards'));

    if (response.statusCode == 200) {
      return json.decode(response.body);
    } else {
      throw Exception('Failed to load wards');
    }
  } catch (e) {
    throw Exception('Network error: $e');
  }
}
```

5.3 Create Data Model

```
// lib/models/facility.dart
import 'package:flutter/material.dart';

class Facility {
  final int id;
  final String name;
  final String? nameMarathi;
  final String? nameHindi;
  final String wardCode;
  final String wardName;
  final double locationLat;
  final double locationLng;
  final String address;
  final String contactNumber;
  final String? emergencyContact;
  final String facilityType;
  final String ownership;
  final bool hasIcu;
  final bool hasVentilator;
  final bool hasOxygenPlant;
  final bool hasAmbulance;
  final int bedsTotal;
  final int bedsAvailable;
  final int icuTotal;
  final int icuAvailable;
  final int ventilatorsTotal;
  final int ventilatorsAvailable;
  final bool oxygenAvailable;
  final DateTime lastUpdated;
  final String bedStatus; // 'good', 'warning', 'critical'
```

```

final bool isRecent;

Facility({
  required this.id, required this.name, this.nameMarathi, this.nameHindi,
  required this.wardCode, required this.wardName, required this.locationLat,
  required this.locationLng, required this.address, required
this.contactNumber,
  this.emergencyContact, required this.facilityType, required this.ownership,
  required this.hasIcu, required this.hasVentilator, required
this.hasOxygenPlant,
  required this.hasAmbulance, required this.bedsTotal, required
this.bedsAvailable,
  required this.icuTotal, required this.icuAvailable, required
this.ventilatorsTotal,
  required this.ventilatorsAvailable, required this.oxygenAvailable,
  required this.lastUpdated, required this.bedStatus, required this.isRecent,
});

factory Facility.fromJson(Map<String, dynamic> json) {
  return Facility(
    id: json['id'],
    name: json['name'],
    nameMarathi: json['name_marathi'],
    nameHindi: json['name_hindi'],
    wardCode: json['ward_code'],
    wardName: json['ward_name'] ?? '',
    locationLat: (json['location_lat'] as num).toDouble(),
    locationLng: (json['location_lng'] as num).toDouble(),
    address: json['address'] ?? '',
    contactNumber: json['contact_number'] ?? '',
    emergencyContact: json['emergency_contact'],
    facilityType: json['facility_type'] ?? '',
    ownership: json['ownership'] ?? '',
    hasIcu: json['has_icu'] ?? false,
    hasVentilator: json['has_ventilator'] ?? false,
    hasOxygenPlant: json['has_oxygen_plant'] ?? false,
    hasAmbulance: json['has_ambulance'] ?? false,
    bedsTotal: json['beds_total'] ?? 0,
    bedsAvailable: json['beds_available'] ?? 0,
    icuTotal: json['icu_total'] ?? 0,
    icuAvailable: json['icu_available'] ?? 0,
    ventilatorsTotal: json['ventilators_total'] ?? 0,
    ventilatorsAvailable: json['ventilators_available'] ?? 0,
    oxygenAvailable: json['oxygen_available'] ?? false,
    lastUpdated: DateTime.parse(json['last_updated']),
    bedStatus: json['bed_status'] ?? 'unknown',
    isRecent: json['is_recent'] ?? false,
  );
}

Color get statusColor {
  switch (bedStatus) {
    case 'good': return Colors.green;

```

```

        case 'warning': return Colors.orange;
        case 'critical': return Colors.red;
        default: return Colors.grey;
    }
}
}

```

Part 6: Screen Recording and Demo Tips

6.1 Local Running Order

1. **Start PostgreSQL** (usually runs automatically as a Windows service).
2. **Start Next.js Development Server:**

```
npm run dev
```

3. **Start Flutter App:**

```
flutter run
```

6.2 Connection Address Memo

Target	Address
Next.js API Local	http://localhost:3000/api/...
Android Emulator to Host	http://10.0.2.2:3000/api/...
iOS Emulator to Host	http://localhost:3000/api/...
Physical Device Access	Replace with PC IP (e.g., http://192.168.1.5:3000/api/...)

6.3 Recording Highlights

- **Show Data Richness:** Switch between different wards and show real-time changes in the hospital list.
- **Show Multi-language Support:** Switch languages in the Flutter app to see localized advisory content.
- **Show Real-time Updates:** Manually edit bed data in the database and refresh the app to see the status change.
- **Show Offline Features:** Demonstrate the ASHA app saving reports when disconnected and auto-syncing when back online.

Conclusion

Congratulations! You now have a **fully functional, data-rich, production-grade** local development environment for Arogya-SMC. This guide not only helps you master the latest tech stack but also shows you how to support a complete smart healthcare system with a realistic data structure.

Good luck iterating on this solid foundation to create a project that impresses the judges! 🚀